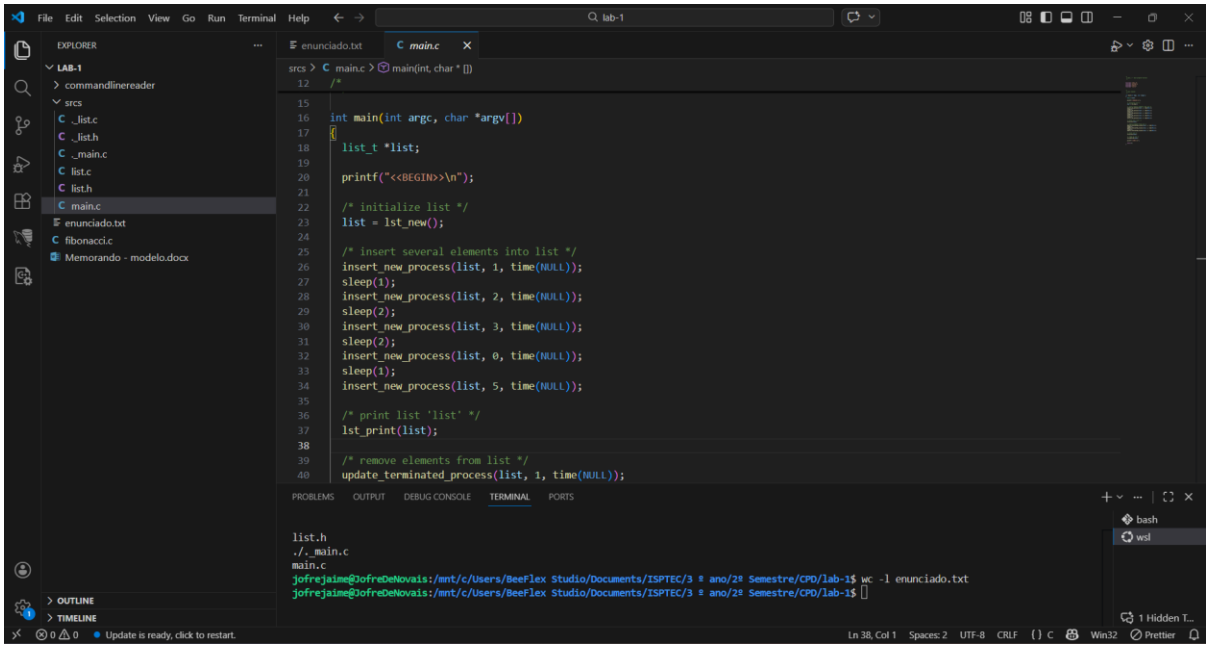
Memorando

De: Jofre Jaime Jamuinda Mutumbungo
Nº de Matrícula: 202302394
Disciplina: Computação Paralela e destrinuida
Assunto: Threads e Processps
Data: 21/03/2026

1. Introdução

O presente memorando descreve as atividades realizadas no Laboratorio 1 da disciplina de Computacao Paralela e Distribuida, com foco no ciclo de desenvolvimento em linguagem C no ambiente Unix (WSL/Ubuntu). O trabalho envolveu compilacao com gcc, depuracao com gdb, analise de falhas por segmentation fault e utilizacao de make para automatizar o processo de construcao.
O objetivo principal foi consolidar competencias praticas de desenvolvimento, depuracao e organizacao do processo de build.



2. Experiências Realizadas

2.1 Preparacao do ambiente

Foi verificada a disponibilidade das ferramentas necessarias (gcc, gdb, make) no Ubuntu via WSL. Durante a preparacao, ocorreram erros de digitacao que foram corrigidos (gbd em vez de gdb, e uso indevido de $ como parte do comando). Apos correcao, o ambiente ficou operacional.

Comandos principais:
sudo apt update
sudo apt install -y gdb
gdb –version

2.2 Geracao do executavel (Secao 3 do enunciado)
Os ficheiros list.c e main.c foram compilados com simbolos de depuracao e ligados no executavel main.

Comandos:
gcc -g -c list.c main.c
gcc -o main list.o main.o
./main

Resultado observado: criacao de list.o, main.o e main, seguida da execucao normal da aplicacao.

```
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/
rcs$ ./main
<<BEGIN>>
Process list with start and end time:
5       Sun Mar 22 23:22:30 2026
        Thu Jan  1 01:00:00 1970
0       Sun Mar 22 23:22:29 2026
        Thu Jan  1 01:00:00 1970
3       Sun Mar 22 23:22:27 2026
        Thu Jan  1 01:00:00 1970
2       Sun Mar 22 23:22:25 2026
        Thu Jan  1 01:00:00 1970
1       Sun Mar 22 23:22:24 2026
        Thu Jan  1 01:00:00 1970
-- end of list.
teminated process with pid: 1
teminated process with pid: 2
teminated process with pid: 5
Process list with start and end time:
5       Sun Mar 22 23:22:30 2026
        Thu Jan  1 01:00:00 1970
0       Sun Mar 22 23:22:29 2026
        Thu Jan  1 01:00:00 1970
3       Sun Mar 22 23:22:27 2026
        Thu Jan  1 01:00:00 1970
2       Sun Mar 22 23:22:25 2026
        Thu Jan  1 01:00:00 1970
1       Sun Mar 22 23:22:24 2026
        Thu Jan  1 01:00:00 1970
-- end of list.
<<END>>
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/
rcs$
```

2.3 Utilizacao do gdb (Secao 4 do enunciado, alinea a-n)

A aplicacao foi analisada com gdb, incluindo breakpoints, execucao passo-a-passo e inspecao de variaveis.

Comandos de referencia:

```
gdb main
b list.c:36
r
p item
p *item
s
n
display *item
```

Resposta as questoes:

- Diferenca entre p item e p *item: p item mostra o endereco (ponteiro), p *item mostra o conteudo apontado.

- Diferenca entre s e n:
s entra nas funcoes chamadas;
n executa a linha sem entrar nas funcoes.

- Utilidade de display *item: atualiza automaticamente a visualizacao do valor a cada passo.

```
rcs$ gdb main
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
--Type <RET> for more, q to quit, c to continue without paging--b list.c:36
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from main...
(gdb) b
No default breakpoint address now.
(gdb) b list.c:36
Breakpoint 1 at 0x129e: file list.c, line 40.
(gdb) r
Starting program: /mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/lab-1/srcs/mai
```

```
40              item = (lst_iitem_t *) malloc (sizeof(lst_iitem_t));
(gdb) p item
$1 = (lst_iitem_t *) 0x5555555596b0
(gdb) p *item
$2 = {pid = 0, starttime = 0, endtime = 0, next = 0x20941}
(gdb) s
0x00007ffff7cad684 in __GI___libc_malloc (bytes=32) at ./malloc/malloc.c:3301
warning: 3301   ./malloc/malloc.c: No such file or directory
(gdb) n
3305    in ./malloc/malloc.c
(gdb) next
3313    in ./malloc/malloc.c
(gdb) n
3316    in ./malloc/malloc.c
(gdb) n
3326    in ./malloc/malloc.c
(gdb) n
3328    in ./malloc/malloc.c
(gdb) n
3329    in ./malloc/malloc.c
(gdb) s
arena_for_chunk (ptr=0x5555555596c0) at ./malloc/arena.c:153
warning: 153    ./malloc/arena.c: No such file or directory
(gdb) display *item
No symbol "item" in current context.
(gdb) display *
A syntax error in expression, near the end of `*'.
(gdb) n
insert_new_process (list=0x5555555596b0, pid=1, starttime=1774218619) at list.c:41
41              item->pid = pid;
(gdb) display *item
1: *item = {pid = 0, starttime = 0, endtime = 0, next = 0x0}
(gdb)
```

Para provocar a falha:
ulimit -c 10000000
./main
Foi observado Segmentation fault.

Na analise de core dump no WSL, verificou-se que o ficheiro gerado em /mnt/c ficou vazio (0 bytes), inviabilizando leitura direta por gdb com core.

Comandos e observacoes:
cat /proc/sys/kernel/core_pattern
ls -lh core*
file core*

core_pattern inicial: |/wsl-capture-crash ...
core.main.<pid>: ficheiro vazio.

```
This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
<<BEGIN>>
Process list with start and end time:
5       Sun Mar 22 23:49:18 2026
        Thu Jan  1 01:00:00 1970
0       Sun Mar 22 23:49:17 2026
        Thu Jan  1 01:00:00 1970
3       Sun Mar 22 23:49:15 2026
        Thu Jan  1 01:00:00 1970
2       Sun Mar 22 23:49:13 2026
        Thu Jan  1 01:00:00 1970
1       Sun Mar 22 23:49:12 2026
        Thu Jan  1 01:00:00 1970

Program received signal SIGSEGV, Segmentation fault.
Download failed: Invalid argument.  Continuing without source file ./time/./time/localtime.c.
__GI_localtime (t=0x8) at ./time/localtime.c:54
warning: 54     ./time/localtime.c: No such file or directory
(gdb) bt
#0  __GI_localtime (t=0x8) at ./time/localtime.c:54
#1  0x00007ffff7cde7ed in ctime (t=<optimized out>) at ./time/ctime.c:27
#2  0x0000555555555357 in lst_print (list=0x5555555596b0) at list.c:63
#3  0x00005555555554a6 in main (argc=1, argv=0x7fffffffd938) at main.c:37
(gdb)
```

A analise da falha foi concluida com execucao dentro do gdb:
run
bt

Backtrace observado:
1) __GI_localtime
2) ctime
3) lst_print (list.c:63)
4) main (main.c:37)

Interpretacao tecnica: a falha ocorre em lst_print, associada ao uso de ponteiro invalido/NULL no acesso aos dados de tempo, conduzindo ao SIGSEGV.

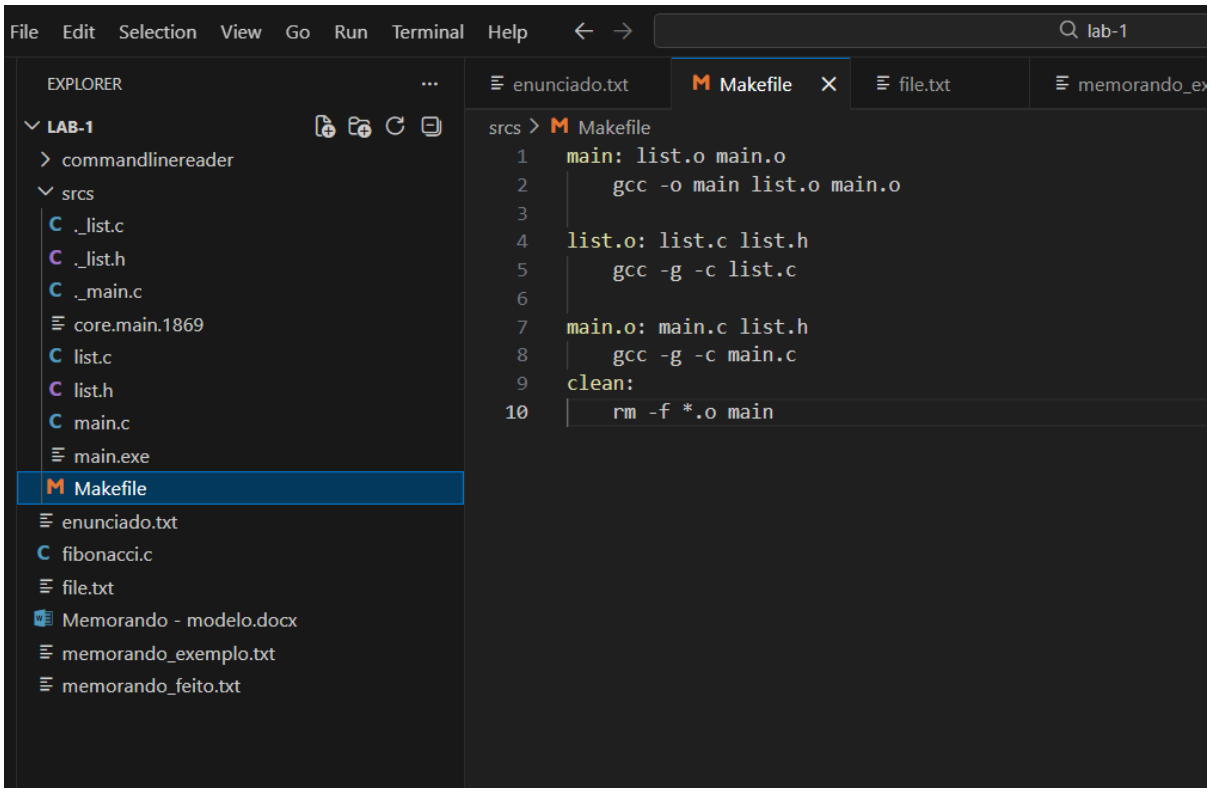
2.4 Utilizacao do make (Secao do enunciado sobre Makefile)

Foi criado um Makefile funcional com regras para main, list.o, main.o e clean.

Estrutura base:
main: list.o main.o
list.o: list.c list.h
main.o: main.c list.h

```
clean:
        rm -f *.o main
```

Interpretacao das alineas:
- make com artefactos atualizados: retorna "main is up to date".
- Ao apagar list.o e correr make: recompila list.o e religa main.
- Com touch main.c e make: recompila main.o e religa main.
- Com alteracao em list.h: recompila os alvos que dependem do header.
- make list.o: atualiza apenas esse alvo; make depois apenas religa se necessario.
- Ao remover dependencia de list.h na regra list.o: alteracoes em list.h deixam de acionar recompilacao correta de list.o.
- Regra clean: alvo clean, sem dependencias, comando rm -f *.o main.
- make clean: remove objetos e executavel; executa sempre que invocada explicitamente.



Desafio - Terminal CPD

O cpd-terminal e um terminal paralelo simples que recebe comandos, cria processos filho com fork + exec, e permite executar varios programas em background sem bloquear o terminal principal.
Tem uma thread monitora que espera a terminacao dos filhos (wait), regista tempos de fim e, no exit, apresenta um relatorio final com pid, estado de saida e tempo de execucao.

Como Executar:

1) Compilar no WSL ou Unix, Linux:
make

2) Executar interativo:

./cpd-terminal

3) Ou executar com ficheiro de comandos:
./cpd-terminal < input.txt

Exemplo de input.txt:
./fibonacci 10000
./fibonacci 10001
./fibonacci 10002
exit

Comandos Suportados

1) pathname [arg1 arg2 ...]
Executa programa em background (maximo 5 argumentos opcionais).

2) exit
Espera todos os filhos terminarem, encerra a thread monitora e imprime relatorio final.

```
jofrejaime@JofreDeNovais:/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/lab-1$
cd "/mnt/c/Users/BeeFlex Studio/Documents/ISPTEC/3 º ano/2º Semestre/CPD/lab-1"
make clean
make
gcc -O2 -o fibonacci fibonacci.c
./cpd-terminal < input.txt
rm -f *.o commandlinereader/*.o cpd-terminal cpdterminal
gcc -Wall -Wextra -g -pthread -c cpd-terminal.c
gcc -Wall -Wextra -g -pthread -c commandlinereader/commandlinereader.c -o commandlinereader/commandli
nereader.o
gcc -Wall -Wextra -g -pthread -o cpd-terminal cpd-terminal.o commandlinereader/commandlinereader.o
Process 2391 started to generate Fibonacci up to 10001.
Process 2390 started to generate Fibonacci up to 10000.
Process 2392 started to generate Fibonacci up to 10002.
Process 2391 finished Fibonacci series at value: 15574651946073070043.
Process 2390 finished Fibonacci series at value: 2788724563990792802.
Process 2392 finished Fibonacci series at value: 18363376510063862845.
===== Process Report =====
pid=2392 status=0 exec_time=1
pid=2391 status=0 exec_time=1
pid=2390 status=0 exec_time=1
```

Notas Importantes (dados no enunciado)

1) Usar fork, exec*, wait, exit, pthread e locks.
2) Leitura de comandos deve usar commandlinereader.
3) Manter estrutura partilhada em lista ligada com pid, tempo de inicio e tempo de fim.
4) Coordenacao por numChildren:
   - numChildren == 0 -> monitora faz sleep(1)
   - caso contrario -> wait
5) No exit, terminar de forma ordeira: principal sinaliza a monitora, espera join, e so depois imprime resultado final.
6) Compilar e testar no WSL/Linux (nao no bash nativo do Windows).

3. Desafios

Como extensao do laboratorio, o desafio requer um terminal paralelo (cpd-terminal) com execucao concorrente e monitorizacao de desempenho. A arquitetura prevista inclui:
- Processo principal para ler comandos com commandlinereader.
- Lancamento de processos filho com fork + exec*.
- Estrutura partilhada em lista ligada para registar pid, tempo de inicio e fim.
- Thread monitora para wait dos filhos e atualizacao de tempos.
- Sincronizacao por pthread_mutex e contador numChildren.
- Encerramento ordeiro no comando exit, com impressao final de pid e tempos de execucao.

4. Repositório GitHub

Jofrejaime/computacao-paralela-e-destribuida
Convite enviado para: joaojdacosta.